第2回 演習・宿題：LoRA で回答形式を学習し、学習前後を比較する

この回は Google Colab（T4 GPU、無料枠）で行います。Codespaces は使いません。

この回のゴール

- 小さなモデルを LoRA で短時間学習させ、学習の前後で **何が変わり、何が変わらないか** を数字と出力で確かめる
- 「形式は学習で身につくが、知らないことへの判断（確認を求めるか、断定するか）までは保証されない」ことを、自分の結果で説明できる
- ファインチューニングが向く場面と向かない場面を、自分の言葉で説明できる

使うノートブック

- notebooks/lora_format_tuning.ipynb
- Colab で開く：https://colab.research.google.com/github/matsuolab/lecture-ai-engineering-2026/blob/main/notebooks/lora_format_tuning.ipynb

架空の講座 FAQ（5 項目）を題材に、`Qwen/Qwen3.5-0.8B` を LoRA で 20 エポック（学習データ 14 件を 20 周、40 ステップ。T4 で 4〜5 分）学習します。望ましい出力は `{"answer": ..., "needs_review": ...}` の JSON 1 行で、FAQ に無い質問には `needs_review: true` で「確認が必要」と答えるのが正解です。評価は **形式遵守率**（JSON の形式を守れたか）と **内容正解率**（`needs_review` と答えの語句が正解と合うか）の 2 つに分けます。

事前準備（任意、5 分）

授業の前にノートブックのセクション 0〜3 を実行し、GPU の割り当て、ライブラリの導入、モデルのダウンロードができることを確かめておくと、当日の詰まりが減ります。Colab はセッションが切れると変数が消えるので、当日はあらためて 0 から実行します。

演習（30分）

1. Colab でノートブックを開き、**ファイル → ドライブにコピーを保存** で自分用にコピーする
2. **ランタイム → ランタイムのタイプを変更 → T4 GPU** を選ぶ
3. セクション 0〜3 を上から実行する（約 5 分）。学習前の表で、**どのキーが欠けているか**、文面は FAQ と合っているかを読む
4. セクション 4 で学習する（4〜5 分）。待ち時間に「LoRA と QLoRA の違い」の説明を読む。`trainable%` と `Training Loss` の推移を確認する
5. セクション 5〜6 で学習後を評価し、学習前と比べる

6. 比較表：`format_rate` と `content_rate` はどう変わったか
7. `退行` ：学習前は通っていたのに学習後に落ちた質問（空が期待値）
8. 残った失敗の型：FAQ に無い質問に別項目を当てて**断定**したか、FAQ にある質問に「記載がありません」と**逃げた**か
9. セクション 6：FAQ にも学習データにも無い質問に、確認を求めたか・断定したか
10. セクション 7 で `result.json` を作る。要点を `lectures/02/answer.md` に書き、作業ブランチ `lec02` から自分の fork の `main` へ PR を出す（answer.md の編集は GitHub の画面上で行ってもよい）。`result.json` は `lectures/02/` に置いて PR に含めてもよい
11. **ランタイム → ランタイムを接続解除して削除**

> Colab の GPU は無料枠で使える時間に限りがあります。終わったら必ずランタイムを削除してください。GPU が割り当てられない、または時間内に学習が終わらないときは、ノートブックの 4b に従い、講師が T4 で実行した結果を見ながら比較と考察を進めます。提出時は `executed_training_myself` を `False` にし、自分で実行できた範囲を answer.md に書いてください。

seed を固定しても、GPU 上の学習は完全には再現しません。隣の人と「残った質問」が違っても異常ではないので、失敗の型で比べてください。

宿題（どれか1つで完了）

レベル	内容
基礎	`CONFIG` の `lora_r`・`lr`・`num_train_epochs` のどれか 1 つを変えて再学習し、`format_rate`・`content_rate`・最終 loss・学習時間を元の設定と並べて記録する。変えたら **ランタイム → セッションを再起動** して上から実行する（学習後のモデルが残っているため）。`result.json` を 2 つ比べる
実装	`FAQ`・`GROUNDED`・`UNGROUNDED` を自分の題材（5 項目程度）に置き換えて学習し、学習前後を比べる。JSON の形式と `needs_review` の設計はそのまま使う。「根拠なし」の質問をどう用意したかも書く
発展	`scripts/make_sft_dataset.py` で講座の用語集から Q&A データ（`messages` 形式の JSONL）を作り、FAQ の代わりに学習させる。評価用の質問（用語集にある語・言い回しを変えた質問・用語集にない語）と正解を自分で用意し、学習前後を比べる。用語集を検索する仕組みと比較して、モデルを学習させる際のメリットとデメリットについて考察する

発展のデータ作成（Codespaces で実行し、できたファイルを Colab にアップロードする）：

```
uv run python scripts/make_sft_dataset.py > glossary_sft.jsonl
```

Colab 側では、各行がノートブックの `to_chat` と同じ `messages` 形式なので、読み込んだ行を `Dataset.from_list` にそのまま渡せば、セクション 4 の `SFTTrainer` でそのまま学習できます。評価（`test` と `gold`）は用語集に合わせて自分で作り直してください。

セクション 8（4bit で読み込んで出力を比べる）は学習をしない観察なので、どのレベルでも結果に添えてかまいません。4bit のまま学習する方法（QLoRA）は、この講座の環境（T4）では動作を確認していないため、宿題の対象外です。

注意

- 学習後は変数 `model` にも adapter が差し込まれています。学習前の出力を取り直すときは `tuned.disable_adapter()`（セクション 6 参照）を使うか、セッションを再起動します
- ノートブックのライブラリは動作確認した版（セクション 0 の `PINS`）に固定しています。版を変えると動かなくなることがあります